

Progettazione e implementazione di una pipeline di processamento di dati relativi a voli aerei

Andrea Ferretti

Sistemi e architetture per big data
Università degli Studi di Tor Vergata
Roma, Italia

Abstract—Il progetto consiste nella realizzazione di una pipeline per il processamento di dati riguardanti voli di compagnie aeree statunitensi dal 1 gennaio 2025 al 30 aprile 2025, determinando analisi statistiche su ritardi e tassi di cancellazione. La pipeline copre l'intero ciclo di vita dei dati grazie all'utilizzo di framework distribuiti: Apache NiFi per la fase di data ingestion, Apache Spark per la fase di processing, HDFS per lo storage di dati, Redis per offrire un layer di cache in memoria e Grafana per il consumo e la visualizzazione dei risultati del processamento. Per la realizzazione del progetto è stato utilizzato Gemini per attività di brainstorming e debugging di codice. La decisione di adottare qualsiasi suggerimento fornito dall'AI è stata valutata personalmente tramite acquisizione di informazioni da forum quali StackOverflow e Cloudera, oppure da documentazione ufficiale.

Index Terms—Big Data, Apache Spark, Data Engineering, HDFS, Apache NiFi

I. INTRODUZIONE

Nell'era dei big data assume una grande rilevanza la capacità di gestire grandi quantità di dati in modo efficiente ed automatizzato. Questo articolo si pone l'obiettivo di applicare principi di data engineering per gestire una pipeline di dati. Il sistema, implementato modalità standalone mediante virtualizzazione basata su container, gestisce diversi aspetti del ciclo di vita del dato: dall'acquisizione iniziale mediante Apache NiFi, all'archiviazione distribuita e fault-tolerant su cluster HDFS, fino al processamento parallelo su Apache Spark.

Oltre a descrivere la costruzione della pipeline, si analizzano le prestazioni che derivano dalle scelte progettuali.

Il documento è strutturato come segue: la Sezione II espone l'architettura del sistema; la Sezione III dettaglia le metodologie di lavoro; la Sezione IV presenta un'analisi critica delle prestazioni, confrontando empiricamente gli approcci RDD e DataFrame; la Sezione V illustra la fase di visualizzazione e interpretazione dei dati processati.

II. ARCHITETTURA DEL SISTEMA

L'infrastruttura proposta è stata interamente containerizzata utilizzando Docker e Docker Compose. In Fig. 1 sono mostrate le fasi del ciclo di vita dei dati e i framework che le gestiscono.

A. Data Ingestion e Pre-processing

La pipeline inizia con una fase di data ingestion, necessaria per estrarre i dati posti su un server remoto in formato `tar.gz` e salvarli su HDFS. Il framework di data ingestion individuato per questo progetto è Apache NiFi, il quale mette

a disposizione dell'utente un'interfaccia grafica semplice e intuitiva per la realizzazione di pipeline di complessità differenti. Il container NiFi ha un volume per memorizzare le pipeline costruite dall'utente e uno in cui sono memorizzati i file di configurazione di HDFS, necessari per poter accedere all'URL del namenode. A tal proposito, sono stati estratti i file `hdfs-site.xml` e `core-site.xml` dall'immagine HDFS utilizzata per questo progetto. In questa fase è inclusa anche una parte di pre-processamento dei dati, con l'obiettivo di effettuare la decompressione dei file, che altrimenti sarebbe gestito in modo sequenziale tramite Spark, e di svolgere una prima fase di pruning dei file per ridurre la banda di rete e lo spazio di storage utilizzati.

B. Strato di persistenza

Lo strato di persistenza dei dati è implementato tramite HDFS. Per consentire una corretta interazione tra HDFS, NiFi e Spark sono stati estratti i file `hdfs-site.xml` e `core-site.xml` dall'immagine HDFS. All'interno dell'`hdfs-site.xml` sono stati configurati il numero di replicazione, mantenuto a 2 a causa delle risorse limitate, e la dimensione del blocco, impostata a 64MB per non avere blocchi eccessivamente frammentati.

C. Processamento de dati

La fase di processamento dei dati è eseguita mediante Apache Spark, il quale consente di eseguire operazioni sui dati in modo parallelo. Spark, utilizzato attraverso il linguaggio di programmazione Python, è utilizzato anche per una seconda fase di preprocessamento dei dati, in cui si eliminano le colonne che non sono di interesse ai fini delle query da eseguire, riducendo ulteriormente la dimensione dei dati che deve essere gestita a runtime. Come tutti i moduli, anche Spark è eseguito in modalità standalone, attraverso un client che sottomette le richieste di processamento a uno Spark Driver, il quale gestisce due nodi worker. Nell'ecosistema Spark è presente anche l'history server, il quale consente di monitorare e memorizzare differenti metriche di esecuzione dei task. Per consentire il corretto funzionamento dell'history server è necessario aggiungere un volume ai worker e al master. Le query implementate tramite Spark sono state scritte sia mediante l'utilizzo degli RDD, sia mediante DataFrame per confrontarne le prestazioni. Tale scelta richiede due modalità di preprocessamento dei dati differenti, in modo da sfruttare

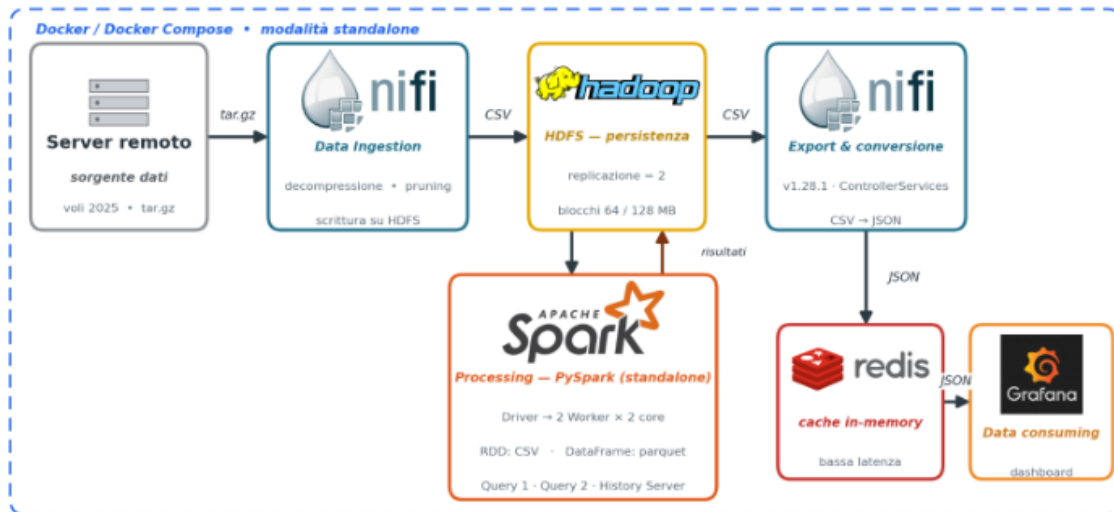


Fig. 1: Architettura logica della data pipeline distribuita. Immagine realizzata con l'ausilio di Intelligenza Artificiale generativa

le ottimizzazioni messe a disposizione dallo Spark engine. Al termine del processamento, i dati sono salvati su HDFS.

D. Layer di storage in-memory

I risultati delle query sono esportati da HDFS su Redis per consentire a sistemi esterni di accedervi con una latenza minore. Anche in questo caso è stato utilizzato NiFi per la gestione del trasferimento dei dati e per la conversione del formato csv in JSON. Tale conversione stata necessaria perché per questo progetto è stata usata la versione 1.28.1 di NiFi, che ha dei limiti di integrazione con Redis. La scelta è stata dettata da stringenti vincoli di compatibilità: le versioni più recenti del framework presentano infatti limitazioni nel supporto nativo ai processori per l'interazione diretta con il file system distribuito Hadoop.

L'integrazione necessaria per la scrittura su Redis è stata ingegnerizzata sfruttando i Controller Services (nello specifico, interfacciandosi con un client cache distribuito). Poiché tale architettura espone un'interfaccia di scrittura chiave-valore generica (*PutDistributedMapCache*), si è resa necessaria un'operazione di trasformazione del dato all'interno della pipeline: i dati in formato CSV sono stati convertiti in formato JSON. Questa strategia di serializzazione ha garantito il mantenimento della struttura delle metriche, rendendole perfettamente compatibili con le logiche di lettura delle dashboard a valle.

E. Data consuming

Nell'ambito di questo progetto, i dati sono consumati da Grafana, un framework di visualizzazione dati distribuito. Esso consente di manipolare i dati e personalizzare il grafico tramite interfaccia grafica, velocizzando così l'implementazione.

III. METODOLOGIA DI ELABORAZIONE DATI

Questa sezione approfondisce le scelte di progettazione e ne spiega le motivazioni.

A. Data Ingestion e Preparazione dello Storage

La prima fase della pipeline prevede l'estrazione dei dati storici relativi ai voli del primo quadrimestre del 2025 (i file `ON_TIME_REPORTING`). Tramite NiFi è stato scaricato e decompresso il file in formato `tar.gz`, per poi estrarre i file al suo interno. Questa operazione di pre-processamento è fondamentale per evitare il collo di bottiglia che si sarebbe creato delegando il compito a Spark, il quale, a causa del formato, sarebbe stato costretto a eseguire i calcoli in modo sequenziale su un unico task. Successivamente, sono stati eliminati i metadati derivanti da computer con sistema operativo MacOS (i file che iniziano con `._`), e file utili per documentare i valori presenti nei file in esame. Questa fase consente di ridurre ulteriormente i dati in transito lungo la pipeline e riduce la frammentazione dei blocchi su HDFS. I dati non sono stati compressi in quanto la dimensione media di un file si aggira intorno ai 60MB, e una eventuale compressione comporterebbe un aumento del tempo di esecuzione. È stato mantenuto il formato CSV poiché la conversione è eseguita in un secondo momento con Spark per sfruttare le ottimizzazioni relative alle diverse API. Inoltre, si è deciso di conservare la suddivisione dei file su base mensile per consentire a Spark di creare più partizioni e lavorare in parallelo. Infatti, avendo a disposizione solo il computer personale, è stato assegnato a ciascun worker 2 core, in modo tale che ogni core sia occupato. Infine, è presente una lieve asimmetria dei dati a causa della diversa durata di ciascun mese ma in tal caso, osservando la dimensione dei file, non vi è un impatto computazionale rilevante.

B. Storage

Il layer di persistenza è stato configurato in modo tale da avere un grado di replicazione pari a 2 e una dimensione dei blocchi pari a 64MB. Tuttavia, avendo file di dimensione superiore a tale dimensione, il processor che comunica con

HDFS (PutHDFS) è stato configurato in modo tale che il dataset sia salvato in blocchi di 128MB, evitando così di utilizzare un blocco aggiuntivo.

C. Logica di Calcolo in PySpark

Per l'elaborazione dei dati sono state progettate specifiche query mirate all'estrazione di statistiche relative ai voli aerei, quali i ritardi medi accumulati (suddivisi per causale) e il tasso di cancellazione, raggruppati per compagnia aerea.

Le query sono state implementate utilizzando due approcci differenti per consentire un confronto prestazionale:

- **Resilient Distributed Dataset (RDD):** L'API di basso livello è stata impiegata definendo manualmente le operazioni di map, filter e le funzioni di riduzione (come *aggregateByKey*), garantendo un controllo più fine sul partizionamento e sullo shuffle di rete. A tal fine, si è deciso di mantenere il formato CSV, in quanto gli RDD non hanno ottimizzazioni che sfruttano il formato colonnare.
- **DataFrame API:** Le interrogazioni sono state riscritte sfruttando l'astrazione di alto livello. Questo approccio ha permesso di ottimizzare l'execution plan al Catalyst Optimizer, il quale ha generato il piano fisico di esecuzione minimizzando le letture ridondanti. Per sfruttare appieno il catalyst optimizer, durante la fase di preprocessing con Spark i dati sono stati trasformati in formato parquet, il quale è ampiamente supportato da Spark e dalle API di alto livello. Infatti, il catalyst optimizer fornisce meccanismi come il predicate pushdown e il partition discovery.

1) *Preprocessing*: La fase di pre-rocessamento in Spark consiste nella rimozione delle colonne del dataset non coinvolte nelle query e nella conversione del formato di salvataggio dei dati. I valori nulli non sono stati imputati o rimossi, ma la loro gestione è posticipata nella fase di implementazione delle query. In particolare, nel caso di RDD si è deciso di assegnare ai valori nulli il valore **None**, un singleton offerto da Python, e di implementare flussi condizionali determinati dalla presenza di questo valore all'interno del record; nel caso dei Dataframe, la gestione è automatizzata e semplificata dalle trasformazioni di aggregazione, che ignorano i valori nulli.

a) *RDD*: nel caso degli RDD si è deciso di mantenere il formato csv perchè queste API non hanno meccanismi di ottimizzazione a livello di lettura. Dunque, la fase di preprocessing ha l'obiettivo di eliminare le colonne che non sono utili per l'esecuzione della query e l'header.

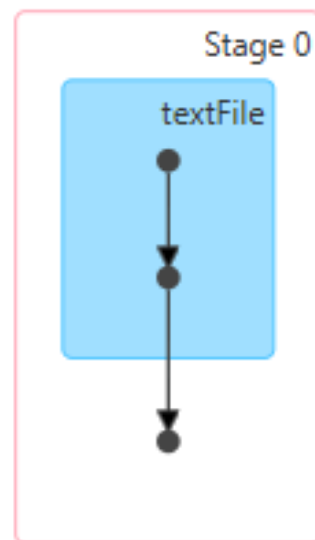


Fig. 2: DAG relativo alla query di preprocessing eseguita tramite RDD

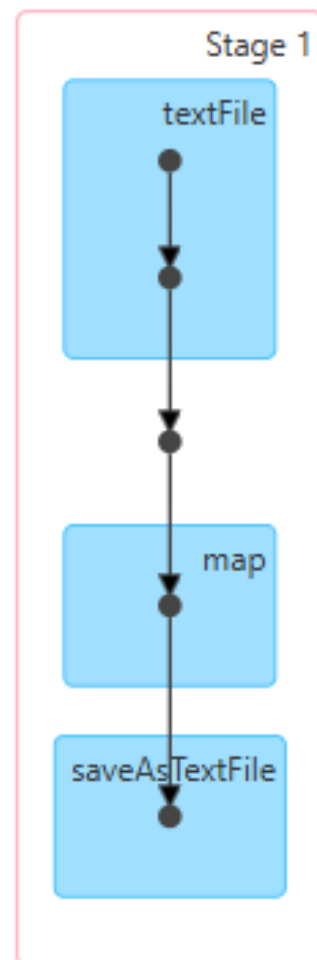


Fig. 3: DAG relativo alla query di preprocessing eseguita tramite RDD

b) *Dataframe*: nel caso dei Dataframe il formato di dati adottato è parquet. In questo caso, oltre alla rimozione delle colonne, si è deciso di inferire manualmente lo schema del dataset poichè lo schema-on-read comporta la scansione dell'intero file, risultando inefficiente.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

def preprocessing():

    spark = SparkSession.builder.appName("SABD_preprocess_df").getOrCreate()

    # Specifico directory input e output
    hdfs_in_dir = "/progetto_sabd/dataset"
    hdfs_parquet_out_dir = "/progetto_sabd/cleaned_dataset/parquet"

    df = spark.read.csv(hdfs_in_dir,
                        header = True,
                        inferSchema = False)

    # Selezione le colonne necessarie per le query e gli assegno il tipo
    processed_df = df.select(
        F.col("MONTH"),
        F.col("OP_UNIQUE_CARRIER"),
        F.col("DEP_DELAY").cast("float"),
        F.col("ARR_DELAY").cast("float"),
        F.col("CANCELLED").cast("float"),
        F.col("DIVERTED").cast("float"),
        F.col("CARRIER_DELAY").cast("float"),
        F.col("WEATHER_DELAY").cast("float"),
        F.col("NAS_DELAY").cast("float"),
        F.col("SECURITY_DELAY").cast("float"),
        F.col("LATE_AIRCRAFT_DELAY").cast("float")
    )

    # Salvo i file su HDFS in formato parquet per query con DataFrame
    processed_df.write \
        .mode("overwrite") \
        .partitionBy("MONTH") \
        .option(key: "compression", value: "LZ4") \
        .parquet(hdfs_parquet_out_dir)
```

Fig. 4: Implementazione preprocessing tramite dataframe

2) *Query 1*: La prima query consente di calcolare la media, il massimo e il minimo del ritardo di partenza dei voli delle compagnie aeree AA (American Airlines) e DL (Delta) su base mensile. Inoltre, calcola per le medesime compagnie il tasso di cancellazione su base mensile.

a) *RDD*: Nell'implementazione tramite RDD si creano delle coppie chiave valore, dove la chiave è costituita dalla compagnia aerea e dal mese. Successivamente, si filtrano le compagnie aeree AA e DL. Per calcolare le statistiche si utilizza la *aggregateByKey*. Questa trasformazione consente di specificare un valore di partenza, una funzione di aggregazione parziale all'interno della partizione e una funzione di aggregazione tra partizioni. Il vantaggio di tale trasformazione è l'esecuzione immediata della funzione di aggregazione parziale, seguita dalla funzione di aggregazione tra partizioni, con una conseguente riduzione della quantità di dati scambiati tra i task tramite shuffle. In questo caso si calcolano il massimo, il minimo, la somma totale del ritardo e del numero di voli che hanno ritardato, e la somma dei voli cancellati e del numero di voli totali. Per calcolare la media del ritardo di partenza e il tasso di cancellazione si ricorre a una *map*. Successivamente, si ordinano le tuple in

ordine lessicografico crescente rispetto alla chiave attraverso una *sortByKey*. Infine, si formattano i dati in formato stringa per salvarli su HDFS.

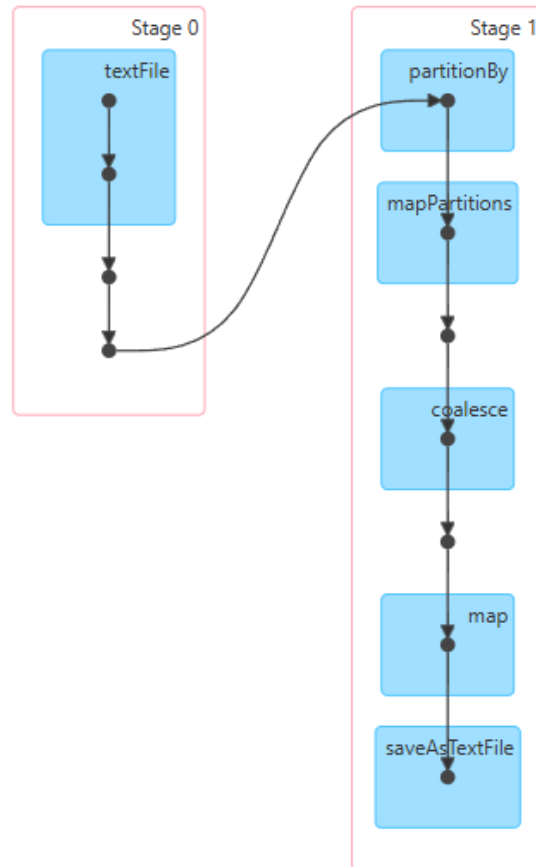


Fig. 5: DAG relativo alla query 1 implementata tramite RDD

b) *Dataframe*: L'implementazione della con i Dataframe si contraddistingue per una maggiore espressività e leggibilità del codice. Il piano logico di esecuzione consiste nell'esecuzione di una *filter* per considerare solamente le righe inerenti alle compagnie aeree AA e DL, una *group by* per partizionare i dati in base alla chiave composta (compagnia aerea, mese) e una serie di *aggregazioni* per calcolare le statistiche richieste. Infine, si raccolgono i dati su un'unica partizione mediante una *coalesce*, si ordinano le righe in ordine lessicografico crescente rispetto alla chiave e si salva il risultato sul file csv. Notare che anche il salvataggio dei dati su file risulta molto semplice poichè la formattazione dei dati è svolta dalle API di Spark.

```

from pyspark.sql import SparkSession
from pyspark.sql import functions as F

def query1_df():
    spark = SparkSession.builder.appName("SABO_query1_df").getOrCreate()

    # Carico il dataset
    hdfs_directory = "/progetto_sabo/cleaned_dataset/parquet"
    df = spark.read.parquet(hdfs_directory)

    # Seleziono le colonne necessarie per la query e converto in rdd
    cols_to_process = ["MONTH", "OP_UNIQUE_CARRIER", "DEP_DELAY", "CANCELLED"]
    final_cols = ["month", "airline", "dep-delay-mean", "dep-delay-min", "dep-delay-max", "cancellation-rate"]

    # Calcolo le statistiche
    result_df = df.select(*cols_to_process) \
        .filter("OP_UNIQUE_CARRIER='AA' OR OP_UNIQUE_CARRIER='DL'") \
        .groupBy("MONTH", "OP_UNIQUE_CARRIER") \
        .agg(
            F.round(F.avg(F.when(F.col("CANCELLED") == 0, F.col("DEP_DELAY"))), (SCALE 2).alias("dep-delay-mean"),
            F.round(F.min(F.when(F.col("CANCELLED") == 0, F.col("DEP_DELAY"))), (SCALE 2).alias("dep-delay-min"),
            F.round(F.max(F.when(F.col("CANCELLED") == 0, F.col("DEP_DELAY"))), (SCALE 2).alias("dep-delay-max"),
            F.round((F.sum("CANCELLED") / F.count("CANCELLED")), (SCALE 2).alias("cancellation-rate")
        ))
    .withColumnRenamed(existing: "MONTH", new: "month") \
    .withColumnRenamed(existing: "OP_UNIQUE_CARRIER", new: "airline") \
    .select(*final_cols) \
    .coalesce(1) \
    .sort("airline", "month", ascending = True)

    # Salvo i dati
    result_df.write \
        .format("csv") \
        .option("key", "header", value: "true") \
        .save("/progetto_sabo/results/dataframe/query1")

```

Fig. 6: Implementazione query 1 tramite dataframe

3) *Query 2*: La seconda query consiste nel calcolare, per ciascuna compagnia, il numero totale di voli che non sono stati cancellati né devianti, i valori medi del ritardo di arrivo e delle cause di ritardo. In questo caso si è deciso di calcolare l'incidenza media delle cause di ritardo solo nel caso in cui il volo abbia ritardato: ciò richiede di ignorare i valori mancanti di questi campi. Successivamente, è necessario considerare le compagnie aeree che hanno almeno 500 voli non cancellati e non devianti e calcolare la classifica delle prime 10 compagnie per ritardo di arrivo medio decrescente.

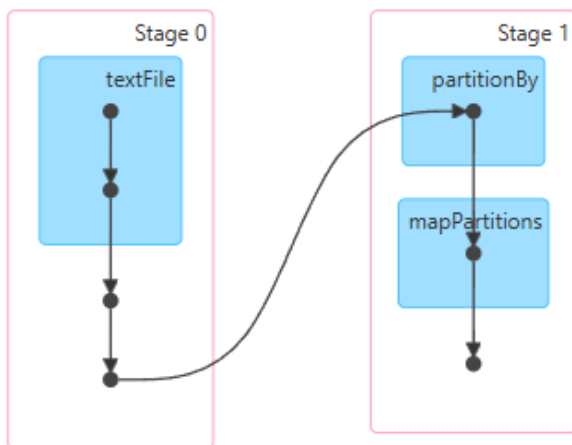


Fig. 7: DAG relativo alla query 2 implementata tramite RDD

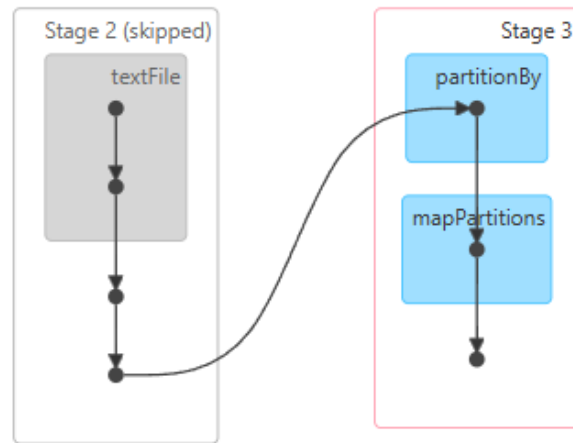


Fig. 8: DAG relativo alla query 2 implementata tramite RDD

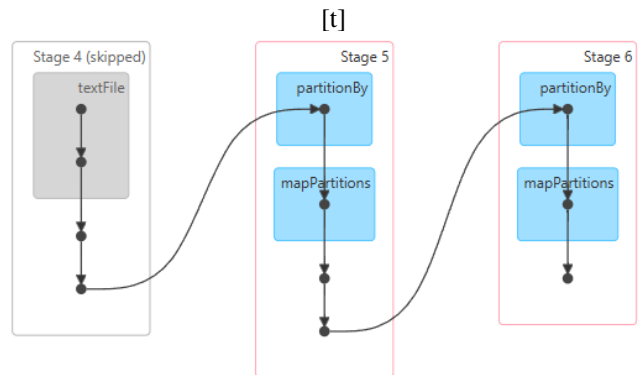


Fig. 9: DAG relativo alla query 2 implementata tramite RDD

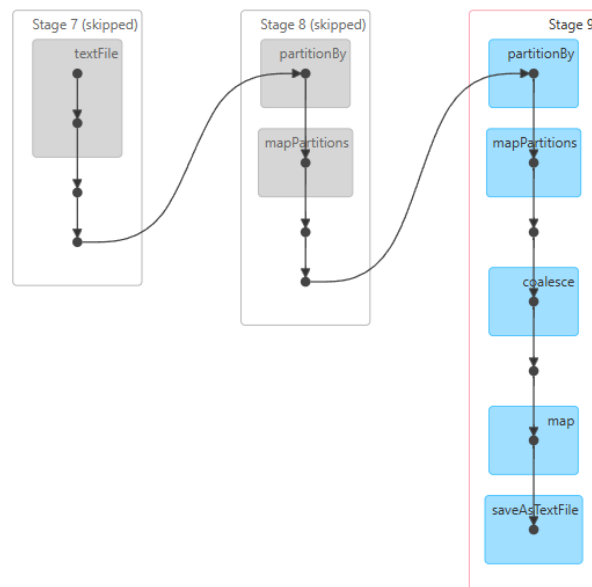


Fig. 10: DAG relativo alla query 2 implementata tramite RDD

a) *RDD*: L'implementazione della query mediante RDD rende difficile la scrittura di codice leggibile. In questo caso la query è stata implementata attraverso una *aggregateByKey* su tutte le statistiche richieste. Il motivo dietro tale scelta è dovuto al fatto che la gestione delle diverse parti della query

mediante trasformazioni opportune sui soli dati inerenti a una delle richieste comporta una serie di letture da HDFS degli stessi dati, oltre a una join finale. In questo caso, quindi, si è sacrificata la leggibilità del codice a favore delle prestazioni. In modo del tutto analogo alla prima query, la `aggregateByKey` è seguita da una `map` per calcolare le medie richieste. Segue una `filter` per filtrare le compagnie aeree che hanno almeno 500 voli che non sono stati nè cancellati nè dirottati. Infine, per stilare la classifica sono state utilizzate le trasformazioni `sortBy` e `zipWithIndex`: la prima consente di ordinare le righe in ordine decrescente di ritardo medio di atterraggio; la seconda assegna un indice incrementale globale a ciascuna tupla, e può essere sfruttata per stilare la classifica senza dover utilizzare l'azione `takeOrdered`. Dunque, dopo aver assegnato gli indici, si selezionano i record con indice minore di 10 (gli indici partono da 0). Infine, si mappa la tupla in modo tale da eliminare l'indice e formattarla opportunamente per essere salvata su HDFS.

b) *Dataframe*: L'implementazione della query mediante dataframe è molto simile all'implementazione della prima query: prima si raggruppano le righe in base alla compagnia aerea attraverso una `groupBy`; successivamente, si eseguono una serie di aggregazioni applicando la trasformazione `avg` per calcolare la media e si utilizza la `count` per calcolare il numero di voli che non sono stati nè cancellati nè dirottati.

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

def query2_df():
    spark = SparkSession.builder.appName("SABD_query2_df").getOrCreate()

    cols_to_process = ["OP_UNIQUE_CARRIER", "CANCELLED", "DIVERTED", "ARR_DELAY", "CARRIER_DELAY", "WEATHER_DELAY",
                      "NAS_DELAY", "SECURITY_DELAY", "LATE_AIRCRAFT_DELAY"]

    # Carico il dataset
    hdfs_directory = "/progetto_sabd/cleaned_dataset/parquet"

    df = spark.read.parquet(hdfs_directory) \
        .select(*cols_to_process)

    result_df = df.groupBy("OP_UNIQUE_CARRIER") \
        .agg(F.count(F.when(F.col("CANCELLED") == 0 & (F.col("DIVERTED") == 0), value: 1)).alias("num_flights"),
             F.round(F.avg("ARR_DELAY"), scale: 2).alias("arr_delay_mean"),
             F.round(F.avg("CARRIER_DELAY"), scale: 2).alias("carrier_delay_mean"),
             F.round(F.avg("WEATHER_DELAY"), scale: 2).alias("weather_delay_mean"),
             F.round(F.avg("NAS_DELAY"), scale: 2).alias("nas_delay_mean"),
             F.round(F.avg("SECURITY_DELAY"), scale: 2).alias("security_delay_mean"),
             F.round(F.avg("LATE_AIRCRAFT_DELAY"), scale: 2).alias("late_aircraft_delay_mean")) \
        .withColumnRenamed(existing: "OP_UNIQUE_CARRIER", new: "carrier")

    top10_df = result_df.filter(F.col("num_flights") >= 500) \
        .coalesce(1) \
        .sort(("cols: 'arr_delay_mean', ascending = False")) \
        .limit(10)

    top10_df.write \
        .format("csv") \
        .option("key: 'header', value: 'true'") \
        .save("/progetto_sabd/results/dataframe/query2")
```

Fig. 11: Implementazione query 2 tramite dataframe

D. Da HDFS a Redis

Il trasferimento dei dati da HDFS a Redis è orchestrato da NiFi, il quale ha il compito di rilevare nuovi file all'interno di una directory HDFS monitorata, eventualmente estrarli e convertirli in formato JSON. Infine, i file sono memorizzati su Redis mediante delle put. La scelta di un framework differente da Spark per eseguire questa fase rende Spark agnostico rispetto al data store utilizzato per mettere a disposizione i risultati delle query agli altri sistemi.

E. Grafana

1) *Query 1*: Per visualizzare i dati mediante Grafana sono state applicate una serie di trasformazioni ai dati in formato JSON:

- si estrae la stringa in formato JSON da Redis
- si traspone la tabella in modo tale da avere una riga per ciascun oggetto JSON
- si ottiene una tabella in cui ciascuna colonna è un campo dell'oggetto JSON
- si selezionano le colonne di interesse
- si traspone la matrice in modo che Grafana possa individuare correttamente gli assi e i valori da graficare

I risultati sono mostrati mediante bar plots verticali, con barre raggruppate per mese.

2) *Query 2*: Per visualizzare i dati sono state applicate le prime quattro trasformazioni utilizzate nella prima query. I dati sono stati visualizzati mediante un bar plot orizzontale per consentire una esposizione più chiara della classifica. Ogni barra mostra la il contributo in valore assoluto e in percentuale di ciascuna causa di ritardo.

IV. VALUTAZIONE DELLE PRESTAZIONI

La valutazione delle prestazioni è stata eseguita in modo isolato su ciascuna fase del ciclo di vita del dato.

A. Valutazione performance della pipeline dei dati

a) *Data ingestion*: Di seguito sono riportati i tempi di esecuzione di ogni processor che costituisce la pipeline di data ingestion dal server remoto a HDFS.

TABLE I: Prestazioni di calcolo: Media e Deviazione Standard dei tempi di esecuzione.

Processor	Tempo di esecuzione(s)
InvokeHTTP	29,58 ± 1,6
CompressContent	3,47 ± 1,04
UnpackContent	7,41 ± 1,64
RouteOnAttribute	0,054 ± 0,027
PutHDFS	9,29 ± 2,69

Dai valori mostrati in tabella è possibile notare che il collo di bottiglia è dato dal processor InvokeHTTP, il quale scarica il file compresso dal server remoto. In questo caso non è possibile migliorare ulteriormente le prestazioni poichè la latenza dipende dalla rete alla quale si è connessi. Neanche CompressContent può essere ottimizzato aggiungendo più task, poichè il formato di compressione del file non può essere processato in modo parallelo. UnpackContent e PutHDFS eseguono più task contemporaneamente.

b) *Spark*: Questa fase della pipeline è stata monitorata utilizzando diverse configurazioni del cluster Spark. Dapprima è stato eseguito un test delle performance con due container worker attivi, assegnando 2 core e 1GB di memoria a ciascun worker e 512 MB di memoria al Driver. Di seguito viene riportata la tabella.

Dalla tabella si evince che i dataframe hanno prestazioni leggermente superiori rispetto agli RDD. Ciò potrebbe essere

TABLE II: Confronto prestazioni operazioni sui dati tramite RDD e Dataframe

Operazione	Tempo di esecuzione(s)
Preprocessing RDD	24,9 ± 3,08
Preprocessing Dataframe	30,9 ± 2,12
Query 1 RDD	13,9 ± 0,94
Query 1 Dataframe	15 ± 0,89
Query 2 RDD	19,6 ± 1,34
Query 2 Dataframe	15,8 ± 0,75

dovuto alle ottimizzazioni messe in atto dal Catalyst Optimizer in fase di lettura del file.

Successivamente, si è deciso di eliminare un container worker e di assegnare a ciascun executor 2 core e 1GB di memoria. Lo Spark Driver mantiene la memoria di 512MB. In questo non si sono confrontate le performance della fase di preprocessing.

TABLE III: Confronto prestazioni operazioni sui dati tramite RDD e Dataframe

Operazione	Tempo di esecuzione(s)
Query 1 RDD	22,4 ± 3,2
Query 1 Dataframe	14,3 ± 0,64
Query 2 RDD	22,4 ± 18,68
Query 2 Dataframe	17,9 ± 3,61

Dai risultati è possibile notare un peggioramento delle prestazioni in entrambe le query eseguite con gli RDD e nella seconda query eseguita mediante Dataframe. D'altra parte, la prima query eseguita con Dataframe ha evidenziato prestazioni leggermente migliori. Ciò è dovuto a una minore disponibilità delle risorse di calcolo del worker.

Infine, è stato selezionato un solo container worker, al quale sono stati assegnati 4 core e 2GB di memoria. Anche in questo caso lo Spark Driver ha dimensione della memoria pari a 512MB, e il preprocessing non è stato considerato.

TABLE IV: Confronto prestazioni operazioni sui dati tramite RDD e Dataframe

Operazione	Tempo di esecuzione(s)
Query 1 RDD	16,9 ± 3,2
Query 1 Dataframe	9,5 ± 0,80
Query 2 RDD	25,83 ± 18,68
Query 2 Dataframe	9,8 ± 0,74

Le prestazioni dei dataframe sono migliorate in modo considerevole, mentre gli RDD hanno prestazioni peggiori rispetto al caso con due container, ma migliori rispetto al caso precedente. Il motivo potrebbe essere legato al fatto che gli RDD devono comunque caricare l'intero file in memoria per poter eseguire le query, mentre i dataframe riducono i dati letti grazie al Catalyst Optimizer, beneficiando quindi delle maggiori risorse a disposizione.

V. DATA VISUALIZATION E CONCLUSIONI

Di seguito sono riportati i grafici relativi alla prima query.



Fig. 12: Grafico relativo al cancellation rate di AA e DL

Il tasso di cancellazione delle due compagnie aeree è molto alto a gennaio, mentre rimane piuttosto costante nei mesi rimanenti. Una possibile causa potrebbe essere il maltempo, che nel mese di gennaio 2025 ha presentato condizioni particolarmente eccezionali in diverse parti degli Stati Uniti, con una combinazione di ondate di freddo, tempeste di neve e fenomeni estremi anche in zone dove sono molto rari.



Fig. 13: Grafico relativo al ritardo medio in partenza di AA e DL

Dal grafico è possibile notare che il ritardo medio in partenza è piuttosto costante nei quattro mesi. Le due compagnie aeree seguono trend opposti, con DL che vede diminuire in modo lineare il ritardo medio in partenza, mentre in AA è possibile notare un aumento progressivo. Tale valore potrebbe essere dovuto al fatto che nel settore aereo statunitense i voli si considerano in ritardo dopo i 15 minuti aggiuntivi rispetto all'orario previsto e pertanto le compagnie aeree progettano gli orari considerando un "buffer" di tale entità.

Di seguito sono riportati i grafici relativi alla seconda query.



Fig. 14: Grafico relativo al ritardo medio in partenza di AA e DL

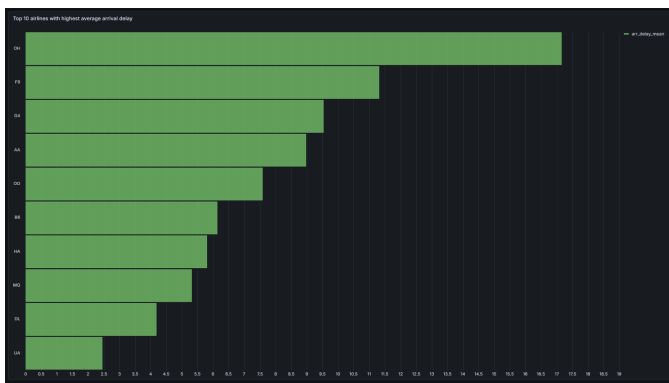


Fig. 15: Grafico relativo al ritardo medio in arrivo sull'intero quadrimestre delle prime 10 compagnie aeree classificate in ordine decrescente

OH è la compagnia aerea con il ritardo maggiore, mentre QA è la compagnia più affidabile per puntualità tra quelle mostrate.

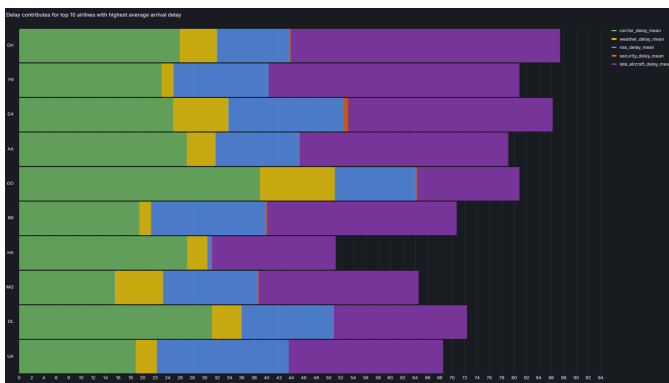


Fig. 16: Grafico relativo ai contributi assoluti delle cause di ritardo in partenza delle prime 10 compagnie aeree in ordine decrescente di ritardo in arrivo

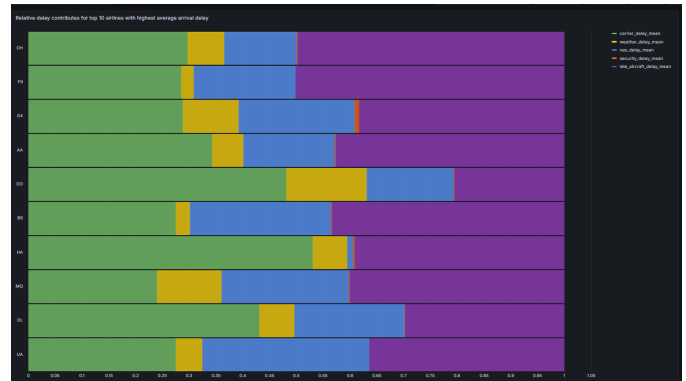


Fig. 17: Grafico relativo ai contributi percentuali delle cause di ritardo in partenza delle prime 10 compagnie aeree in ordine decrescente di ritardo di arrivo

Dal grafico è possibile notare che il ritardo dovuto a possibili minacce di sicurezza è quasi inesistente in tutte le compagnie, dato che questi eventi sono rari. Il contributo maggiore è dato dal ritardo dell'aeromobile in arrivo e dal ritardo dovuto a problemi logistici o operativi della compagnia aerea. I ritardi dovuti al maltempo e al sistema di gestione del traffico aereo sono presenti in quantità moderata.